

JOB PORTAL MANAGEMENT SYSTEM

School of Computing

Bachelor of Technology

in

Computer Science & Engineering

By

P.NIVETHITHA (VTU24312)

10211CS224

Full Stack Application Development

Slot : E2-B8



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

CHENNAI 600 062, TAMILNADU, INDIA

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “**Job Portal Management System**” by **P.Nivethitha (VTU24312)** has been carried out in Winter semester of Academic year 2025-2026 (WS2526).

Signature of Faculty

Dr.N.V.S.Natteshan,

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

Signature of Head of the Department

Dr. M.Kavitha

Professor & Head

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

P.NIVETHITHA

Date: 06/05/2026

ABSTRACT

JobSpark is a full-stack Job Portal Management System developed using Spring Boot, Spring Security, Spring Data JPA, Thymeleaf, and MySQL. The system bridges the gap between job seekers and employers by providing a unified platform for job discovery, application management, and candidate evaluation. The application supports three distinct user roles — Student (Job Seeker), Employer, and Admin — each with a dedicated dashboard and role-based access control enforced through Spring Security.

Students can register, build profiles, upload resumes, search and filter job listings, apply with cover letters, track application statuses in real time, bookmark jobs, and withdraw pending applications. Employers can post, edit, and delete job listings with expiry dates, view all applicants, read cover letters, and shortlist or reject candidates — with automated email notifications sent to students upon status changes. The Admin dashboard provides a complete platform overview including user distribution charts, application statistics, and full user management.

The frontend is built with HTML5, CSS3, Thymeleaf, and JavaScript ES6, featuring a glassmorphism UI design, animated particle backgrounds, Chart.js analytics dashboards, 3D card effects, and a fully responsive layout. All data is persistently stored in a MySQL relational database with Hibernate-managed entity relationships. The application runs on embedded Apache Tomcat and requires no paid hosting or cloud infrastructure.

Keywords: Spring Boot, Spring Security, Spring Data JPA, Thymeleaf, MySQL, Hibernate, Role-Based Access Control, BCrypt, Chart.js, Full Stack Development

LIST OF FIGURES

1.1	Framework of the JobSpark Application.....	3
3.1	Data Flow Diagram of JobSpark Application	8
5.1	JobSpark Homepage with Particle Animation Hero Section	14
5.2	Login Page with Split-Screen Glassmorphism Design	15
5.3	Admin Dashboard with Platform Overview and Chart.js Analytics	16
5.4	Student Dashboard with Application Statistics and Charts.....	17
5.5	Employer Dashboard with Job Management Table and Analytics	18

LIST OF TABLES

2.1	Comparison of Job Portal Platforms	5
3.1	Database Tables and Key Columns	9
5.1	System Performance and Feature Analysis.....	19
8.1	Mapping of Project to Complex Engineering Problem (CEP).....	28
8.2	Mapping of Project to Complex Engineering Activities (CEA) .	29
8.3	SDG Mapping for the Project	30

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
BCrypt	Blowfish Crypt Password Hashing Algorithm
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSRF	Cross-Site Request Forgery
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JPQL	Java Persistence Query Language
JS	JavaScript
MVC	Model View Controller
MySQL	My Structured Query Language
ORM	Object Relational Mapping
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol

SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction.....	1
1.2 Aim of the Project.....	1
1.3 Scope of the Project.....	2
1.4 Framework.....	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6
3.2 Backend Implementation.....	7
3.3 Database Implementation.....	8
4 METHODOLOGIES	10
4.1 Project Architecture	10
4.2 DataFlow Diagram.....	10
4.3 Module 1: User Interface Module.....	11

4.4	Module 2: API and Backend Module	11
4.5	Module 3: Database Module.....	12
5	RESULTS AND DISCUSSIONS	13
6	CONCLUSION AND FUTURE ENHANCEMENTS	20
6.1	Conclusion	20
6.2	Future Enhancements	20
7	SOURCE CODE	22
7.1	Source Code.....	22
8	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	27
8.1	Mapping of the Project to Complex Engineering Problem (CEP)	27
8.2	Mapping of the Project to Complex Engineering Activities (CEA)	29
8.3	SDG Mapping.....	30
	References	30

Chapter 1

INTRODUCTION

1.1 Introduction

Managing job applications and recruitment through manual methods is often inefficient and prone to errors. To address this, a full-stack web application named **JobSpark** is developed to provide a simple and organised platform for job discovery, application management, and candidate evaluation. The system enables students (job seekers) to view job listings, register, apply with cover letters, and track their application status in real time, while helping employers post jobs, manage applicants, and shortlist candidates efficiently. By leveraging Spring Boot, Spring Security, Spring Data JPA, Thymeleaf, and MySQL, the application ensures a secure, responsive, and user-friendly experience, making the overall recruitment process faster, more reliable, and more transparent.

1.2 Aim of the Project

The aim of this project is to design and develop a full-stack Job Portal Management System that efficiently connects job seekers and employers through a secure, role-based web platform. The system focuses on simplifying the process of job discovery, application management, and candidate evaluation by providing a user-friendly interface, ensuring accurate data handling, and reducing manual effort for both students and employers. It also aims to provide automated email notifications, analytics dashboards, and complete administrative

oversight through a dedicated admin panel.

1.3 Scope of the Project

The scope of JobSpark encompasses the following functional areas. Students can register, login, upload resumes, apply for jobs with cover letters, track real-time application status (Pending, Shortlisted, Rejected), withdraw pending applications, and bookmark interesting jobs. Employers can register, post and manage job listings with expiry dates, view all applicants, read cover letters through a modal popup, and shortlist or reject candidates with automated email notifications. The Admin panel provides a complete platform overview with analytics charts, application management, and user deletion capabilities. Future enhancements may include REST API integration, JWT authentication, AI-based job recommendations, cloud deployment, and a dedicated mobile application.

1.4 Framework

The proposed Job Portal Management System is developed using the **Spring Boot** framework — a powerful Java-based framework for building production-ready web applications. Spring Boot provides an embedded Apache Tomcat server, auto-configuration, and starter dependencies that reduce boilerplate code. It utilises a component-based MVC architecture with distinct Controller, Service, and Repository layers, allowing the application to be divided into reusable and maintainable modules such as user management, job management, and application processing. **Spring Security** handles role-based authentication and BCrypt password encryption. **Thymeleaf** serves as the server-side template engine rendering dynamic HTML pages. **MySQL** with Spring Data JPA manages all data storage through Hibernate ORM. **Chart.js** provides real-time analytics on dashboards, making it suitable for handling job listings, user profiles, application tracking, and email notifications.

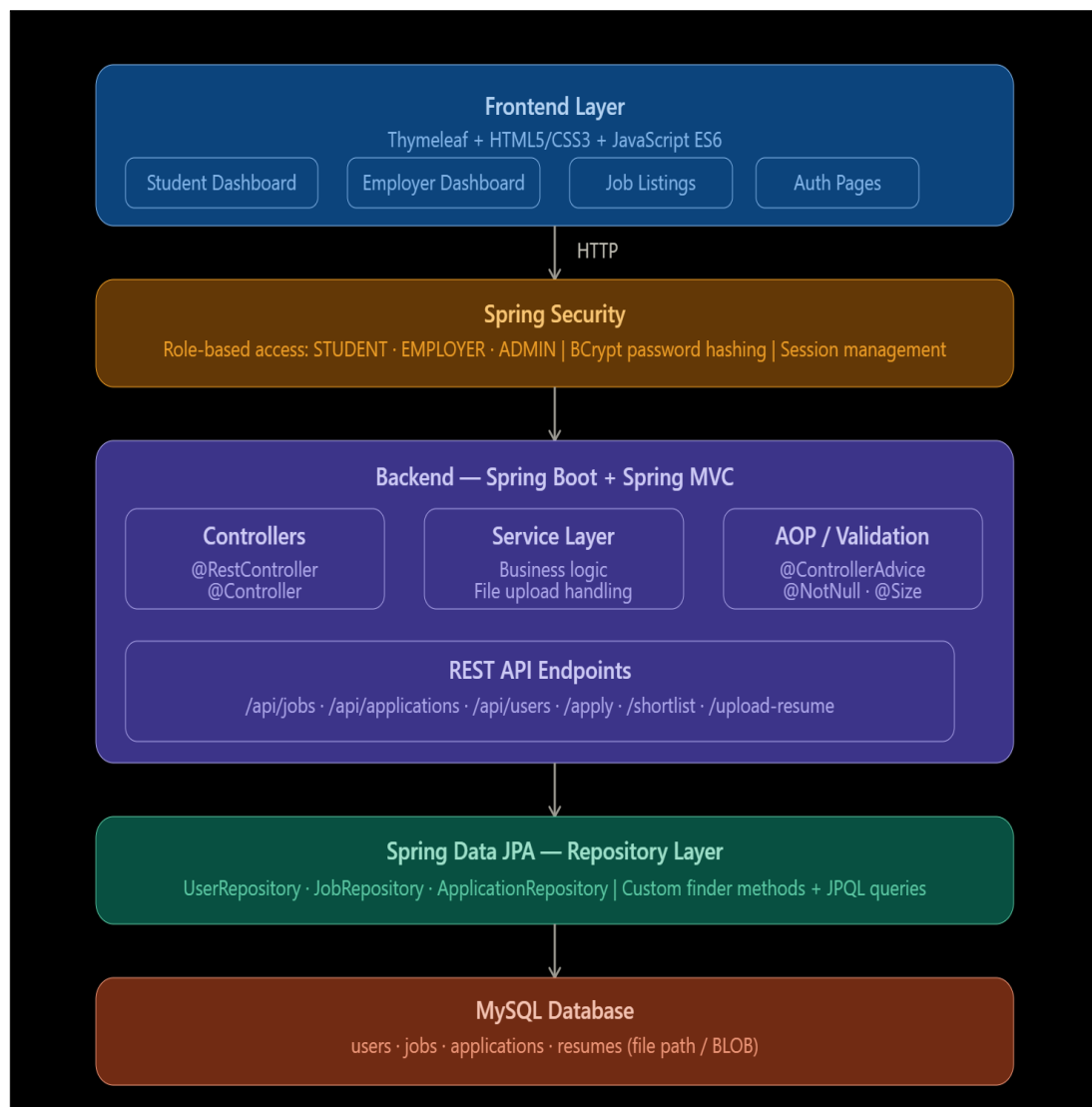


Figure 1.1: Framework of the JobSpark Application

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In the current market, several online job portal platforms have been developed to simplify job discovery and recruitment processes. One of the most prominent platforms is **LinkedIn** (<https://www.linkedin.com>), which provides professional networking along with job listings, AI-based recommendations, recruiter tools, and skill endorsements. However, many premium features require paid subscriptions and the platform can be overwhelming for fresh graduates.

Similarly, **Naukri.com** (<https://www.naukri.com>) is India's leading job portal targeting experienced professionals and freshers. It offers resume building, job alerts, and employer dashboards but lacks real-time application status tracking visible to candidates, and its interface contains many advertisements.

Global platforms like **Indeed** (<https://www.indeed.com>) aggregate job listings from multiple sources. While their search capability is strong, they do not provide employers with built-in applicant management tools or analytics dashboards. **Internshala** (<https://www.internshala.com>) caters specifically to students but is limited to internships and does not support full-time job lifecycle management. **Freshersworld** provides job listings for freshers but lacks an integrated employer dashboard and cover letter management.

This survey highlights that while current applications are functional, there is scope for a simplified, cost-effective, and feature-complete solution. The proposed Spring Boot-based JobSpark application addresses these needs by combining all required features at no cost with real-time tracking and a complete admin panel.

Table 2.1: Comparison of Job Portal Platforms

Feature	LinkedIn	Naukri	Indeed	Internshala	JobSpark
Role-Based Access	Yes	Partial	No	Partial	Yes
Real-Time Tracking	No	No	No	Partial	Yes
Email Notification	Yes	Yes	No	Yes	Yes
Analytics Dashboard	Yes	No	No	No	Yes
Resume Upload	Yes	Yes	Yes	Yes	Yes
Job Expiry Date	Yes	Yes	Yes	No	Yes
Cover Letter View	No	No	No	No	Yes
Application Withdraw	No	No	No	No	Yes
Job Bookmarks	Yes	Yes	Yes	Yes	Yes
Admin Control Panel	No	No	No	No	Yes
Free to Use	Partial	Partial	Yes	Yes	Yes

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The development environment is set up using Eclipse IDE for Enterprise Java Developers with the Spring Tools 4 plugin. The frontend is built using HTML5, CSS3, JavaScript ES6, and Thymeleaf as the server-side template engine. External libraries loaded via CDN include Chart.js for dashboard analytics and the Inter font from Google Fonts. The application is tested using Google Chrome at `http://localhost:8080`.

3.1.2 Structure of the Project

The frontend follows a modular Thymeleaf template structure organised into subdirectories: `templates/auth/` for login and register pages, for student dashboard, applications, profile, saved jobs, and change password pages, `templates/employer/` for employer dashboard, post-job, applicants, edit-job, profile, and change password pages, `templates/admin/` for the admin dashboard, `templates/jobs/` for browse and detail pages, and for the reusable navbar. Styling is handled through a global `style.css` using CSS custom properties for the purple glassmorphism theme. The `main.js` file provides 15 interactive features including loading screen, custom cursor, scroll animations, counter animations, 3D tilt on job cards, confetti for short-

listed students, canvas particle background, and toast notifications.

3.1.3 Execution Procedure

To run the application, first ensure MySQL is running. The Spring Boot application is started by right-clicking `JobportalApplication.java` in Eclipse and selecting Run As → Spring Boot App. Once the console shows *Started JobportalApplication in X seconds*, the application can be accessed at `http://localhost:8080`.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend environment is configured using Java 17 and Spring Boot 3.2.0 with Maven as the build tool. The project is created using Spring Initializr (<https://start.spring.io>). A MySQL 8 database named `jobportal-db` is used. Key Maven dependencies include: `spring-boot-starter-web`, `spring-boot-starter-security`, `spring-boot-starter-data-jpa`, `spring-boot-starter-thymeleaf`, `spring-boot-starter-mail`, `spring-boot-starter-validation`, `mysql-connector-j`, and `lombok`.

3.2.2 Structure of the Project

The backend follows a layered MVC architecture. The `model` package contains JPA entity classes (`User`, `Job`, `Application`, `SavedJob`) and enum types. The `repository` package contains Spring Data JPA interfaces. The `service` package contains business logic classes including `UserService`, `JobService`, `ApplicationService`, `EmailService`. The `controller` package contains `AuthController`, `JobController`, and `Dashboard`. The `security` package contains `SecurityConfig`.

3.2.3 Execution Procedure

The backend is executed by first starting the MySQL service. The `jobportal-db`

database is auto-created on first run via `createDatabaseIfNotExist=true`. Hibernate automatically creates all required tables through `ddl-auto=update`. Once the server is active, all data is successfully stored in MySQL and can be retrieved using queries such as `SELECT * FROM applications;`.

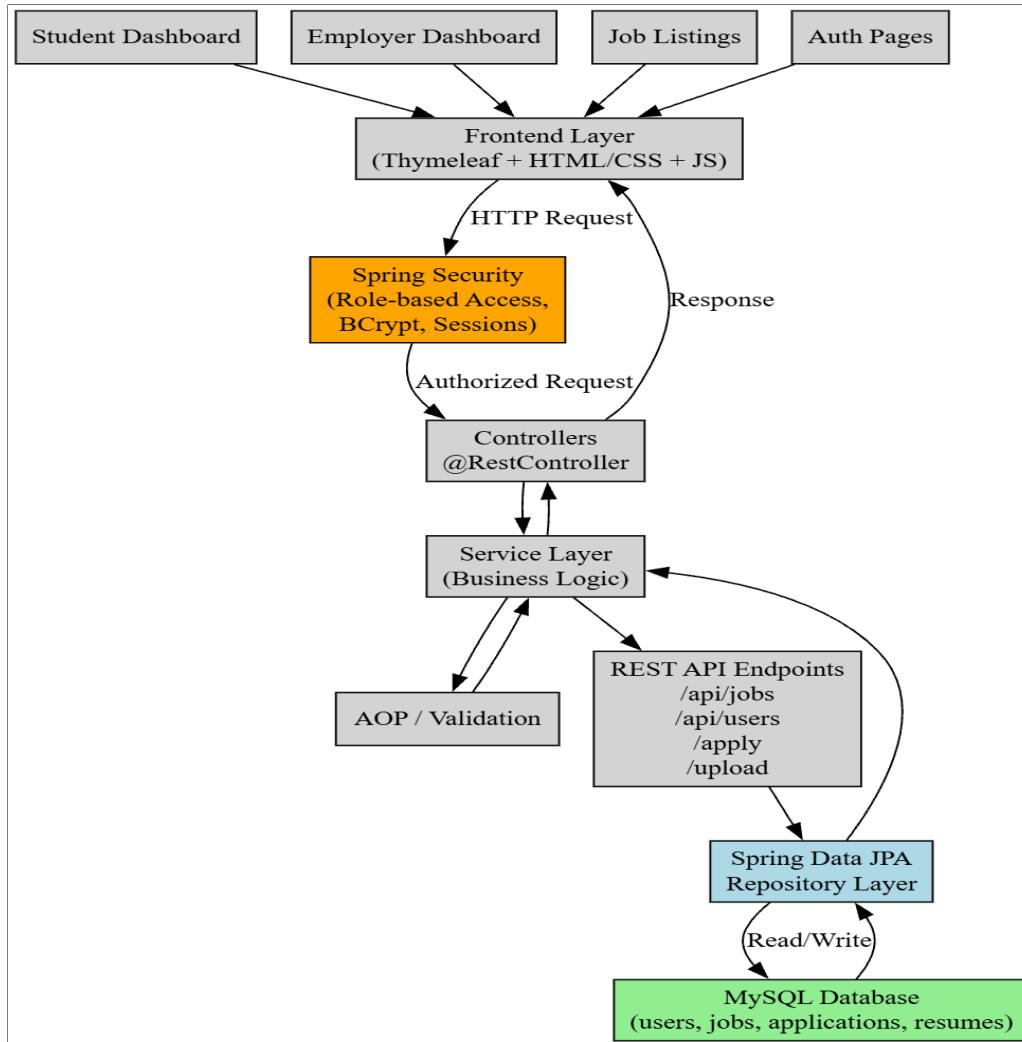


Figure 3.1: Data Flow Diagram of JobSpark Application

3.3 Database Implementation

3.3.1 Database Configuration

The database is configured using MySQL to store and manage user, job, and application data. A database named `jobportal-db` is auto-created using the `createDatabaseIfNotExist=true` parameter in `application.properties`.

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/
```

```

2  jobportal_db?createDatabaseIfNotExist=true
3  &useSSL=false&serverTimezone=UTC
4  &allowPublicKeyRetrieval=true
5  spring.datasource.username=root
6  spring.datasource.driver-class-name=
7    com.mysql.cj.jdbc.Driver
8  spring.jpa.hibernate.ddl-auto=update
9  spring.jpa.show-sql=true
10 spring.mail.host=smtp.gmail.com
11 spring.mail.port=587

```

Listing 3.1: application.properties — Database Configuration

3.3.2 Schema Structure

The database schema consists of four tables. The central entity is `users` which stores all registered users of every role. The `jobs` table references `users` through a foreign key `employer-id`. The `applications` table holds `job-id` and `applicant-id` as foreign keys. The `saved-jobs` table holds `student-id` and `job-id` for the bookmark feature. All relationships are managed using `@OneToMany`, `@ManyToOne`, and `@JoinColumn` JPA annotations.

3.3.3 Tables Created

Table 3.1: Database Tables and Key Columns

Table	Key Columns
users	id, full_name, email, password, role, skills, company_name, resume path, created at
jobs	id, title, description, category, job type, salary, location, experience, status, expiry_date, employer_id
applications	id, cover_letter, status, applied_at, job_id, applicant_id
saved-jobs	id, saved_at, student_id, job_id

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a three-tier layered architecture consisting of presentation, application, and database layers. The **Presentation Layer** includes the frontend built with HTML5, CSS3, Thymeleaf, and JavaScript ES6. The **Application Layer** consists of the Spring Boot backend handling HTTP requests, Spring Security enforcing role-based access, service classes managing business logic, and Spring Mail handling asynchronous email dispatch. The **Database Layer** uses MySQL with Hibernate ORM to store all users, jobs, applications, and saved jobs.

The frontend communicates with the backend through Thymeleaf form submissions and Spring MVC controller mappings. Spring Security intercepts every request to validate authentication and role authorisation before routing to the appropriate controller. After successful login, `CustomAuthSuccessHandler` redirects students to `/student/dashboard`, employers to `/employer`, and admins to `/admin/dashboard`.

4.2 DataFlow Diagram

The data flow begins when the user accesses the web application and views the homepage with latest job listings. The frontend sends a request to to

fetch job information from the database. When a student fills the application form with a cover letter and submits it, the data is validated and sent to `ApplicationService.apply()`. The service checks for duplicate applications, creates a new `Application` entity with `PENDING` status, and persists it to the `applications` table. When an employer changes the status to `SHORTLISTED` or `REJECTED`, `ApplicationService.updateStatus` persists the change and triggers `EmailService` using `@Async` to send an HTML email via Gmail SMTP in a background thread.

4.3 Module 1: User Interface Module

This module handles the design and interaction of the entire application. It includes the homepage with particle animation hero section, job browse page with search and filter, student dashboard with Chart.js doughnut and bar charts, employer dashboard with job management, and admin dashboard with complete platform overview. It ensures a user-friendly glassmorphism experience using CSS3 variables, JavaScript ES6 for 3D tilt effects, counter animations, confetti for shortlisted students, and toast notifications replacing plain HTML alerts.

4.4 Module 2: API and Backend Module

This module manages communication between the frontend and the database. It includes Spring MVC controller endpoints such as `GET /jobs/browse` for fetching job listings, `POST /student/apply/{id}` for processing applications, `POST /employer/post-job` for creating job postings, and `POST /employer/application/{id}/status` for updating application status. The backend validates all input data using `@Valid`, `@NotBlank`, and `@Email` annotations, and handles business logic through dedicated service classes.

4.5 Module 3: Database Module

This module is responsible for all data storage and management through the MySQL database with four tables: `users`, `jobs`, `applications`, and `saved_jobs`. Data integrity is ensured through JPA entity relationships, efficient storage through Hibernate ORM, and quick retrieval through Spring Data JPA custom JPQL queries and finder methods. All tables are automatically created and updated through `ddl-auto=update` with no manual SQL DDL scripts required.

Advantages of this Project

a) Real-Time Application Tracking: Students instantly see whether their application is Pending, Shortlisted, or Rejected on their dashboard with colour-coded badges — no manual follow-up or external communication needed.

b) Automated Email Notifications: The system automatically sends beautifully formatted HTML emails when an employer changes an application status, using asynchronous Spring Mail with Gmail SMTP so it never slows the user interface.

c) Role-Based Security: Spring Security enforces strict URL-level access control ensuring students cannot access employer pages and vice versa, with BCrypt encryption protecting all passwords at rest in the database.

Disadvantages of this Project

The project currently depends on a local server setup which limits public accessibility. Advanced features such as AI-based recommendations, cloud resume storage, and mobile application support are not yet implemented. The system is designed for local deployment and may require optimisation for larger cloud-scale deployments.

Chapter 5

RESULTS AND DISCUSSIONS

The developed Job Portal Management System was successfully implemented and tested across all three user roles. The application allows students to view job listings, apply for jobs, and track application status efficiently. The frontend provides a glassmorphism user-friendly interface, while the Spring Boot backend processes all requests and stores data persistently in the MySQL database. The system was verified with multiple users and all data was correctly stored and retrieved. The application ensures proper validation and displays confirmation messages, toast notifications, and email alerts upon successful operations.

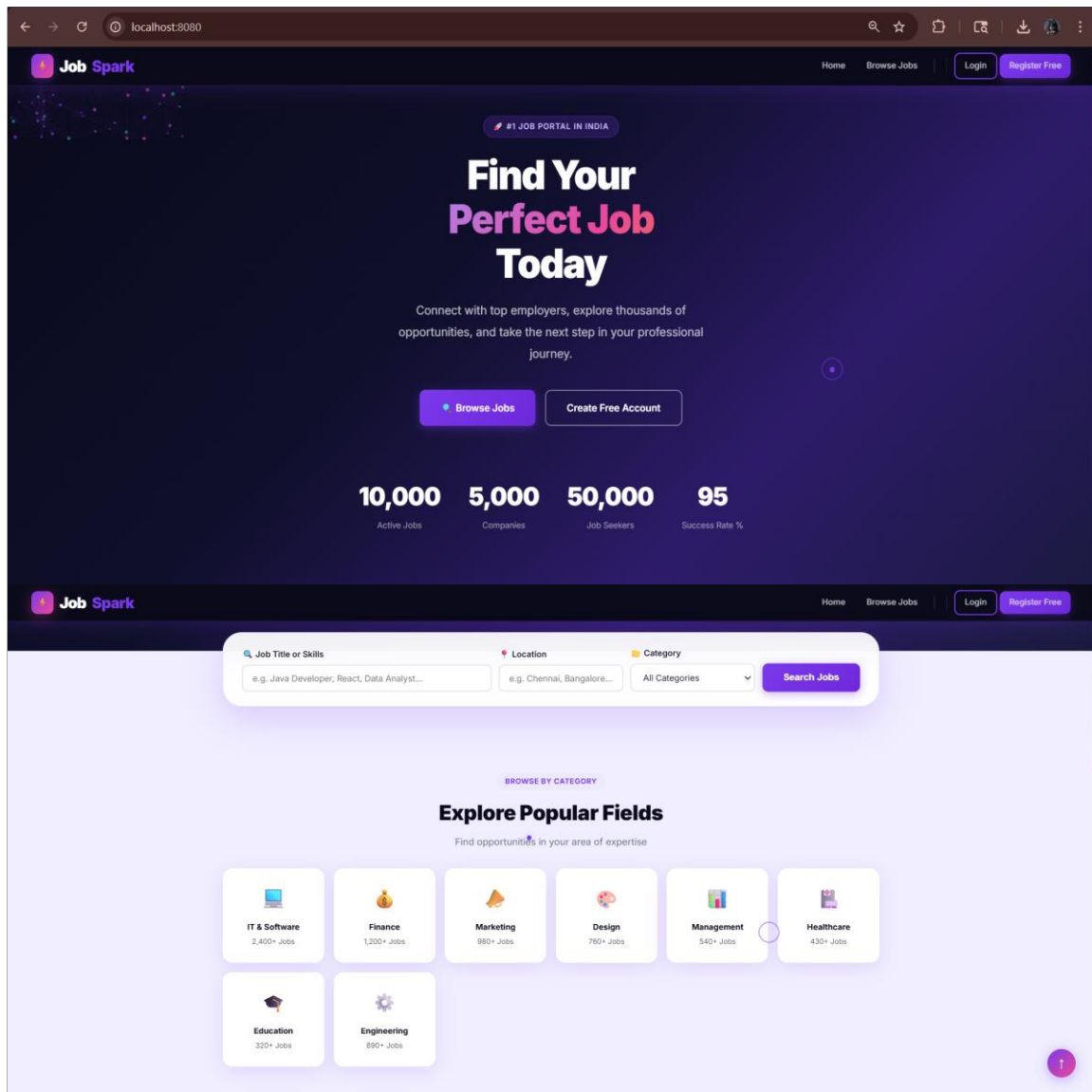


Figure 5.1: JobSpark Homepage with Particle Animation Hero Section

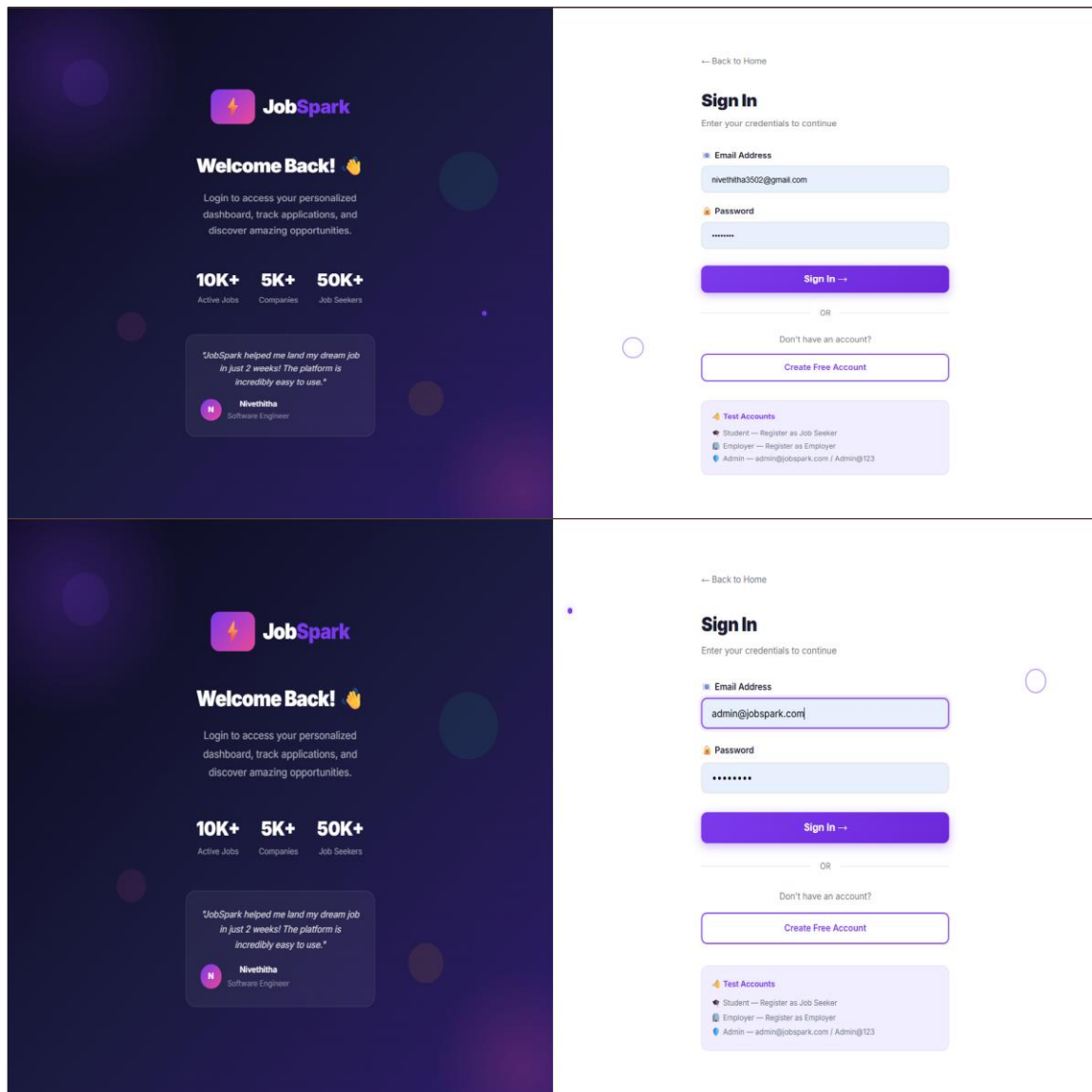


Figure 5.2: Login Page with Split-Screen Glassmorphism Design

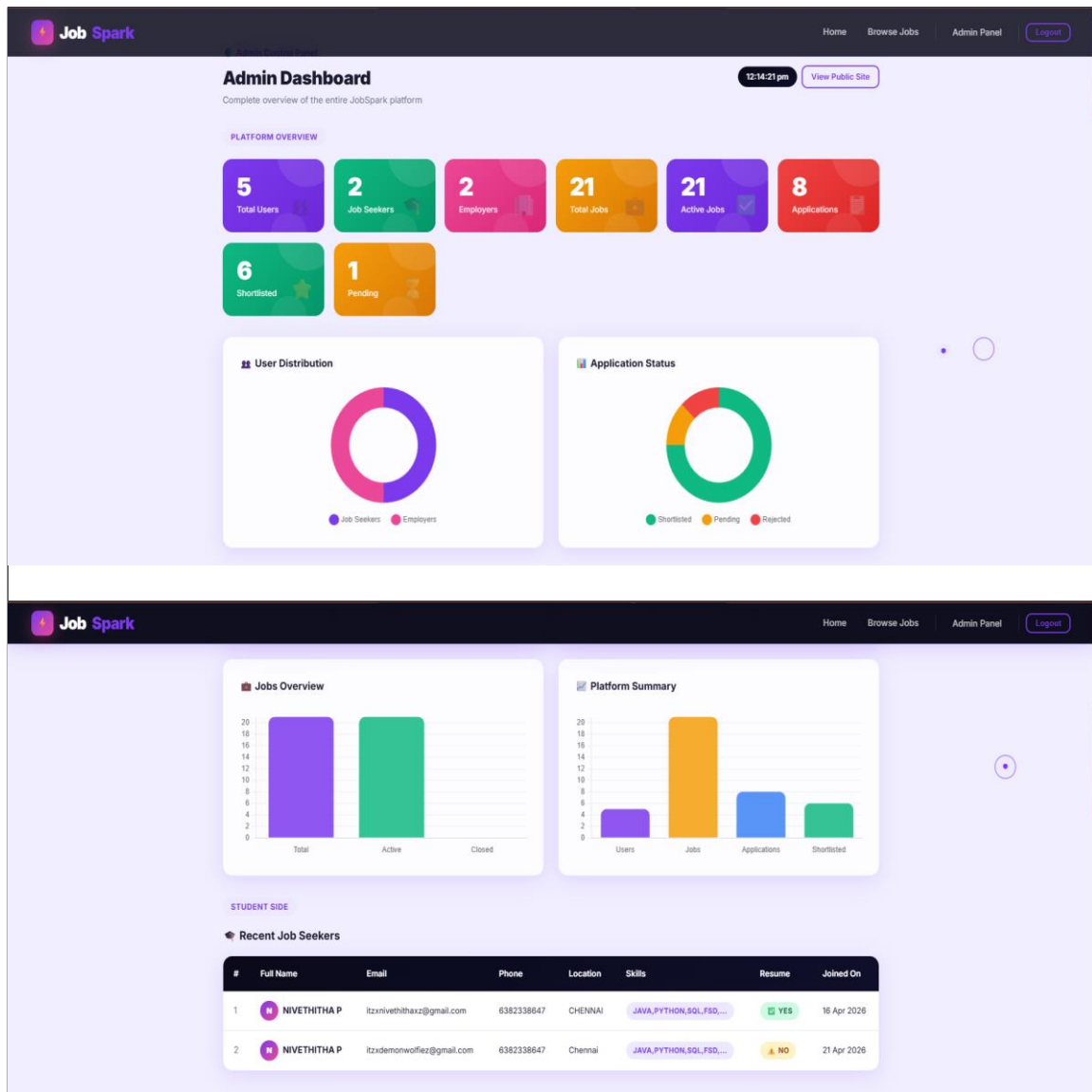


Figure 5.3: Admin Dashboard with Platform Overview and Chart.js Analytics

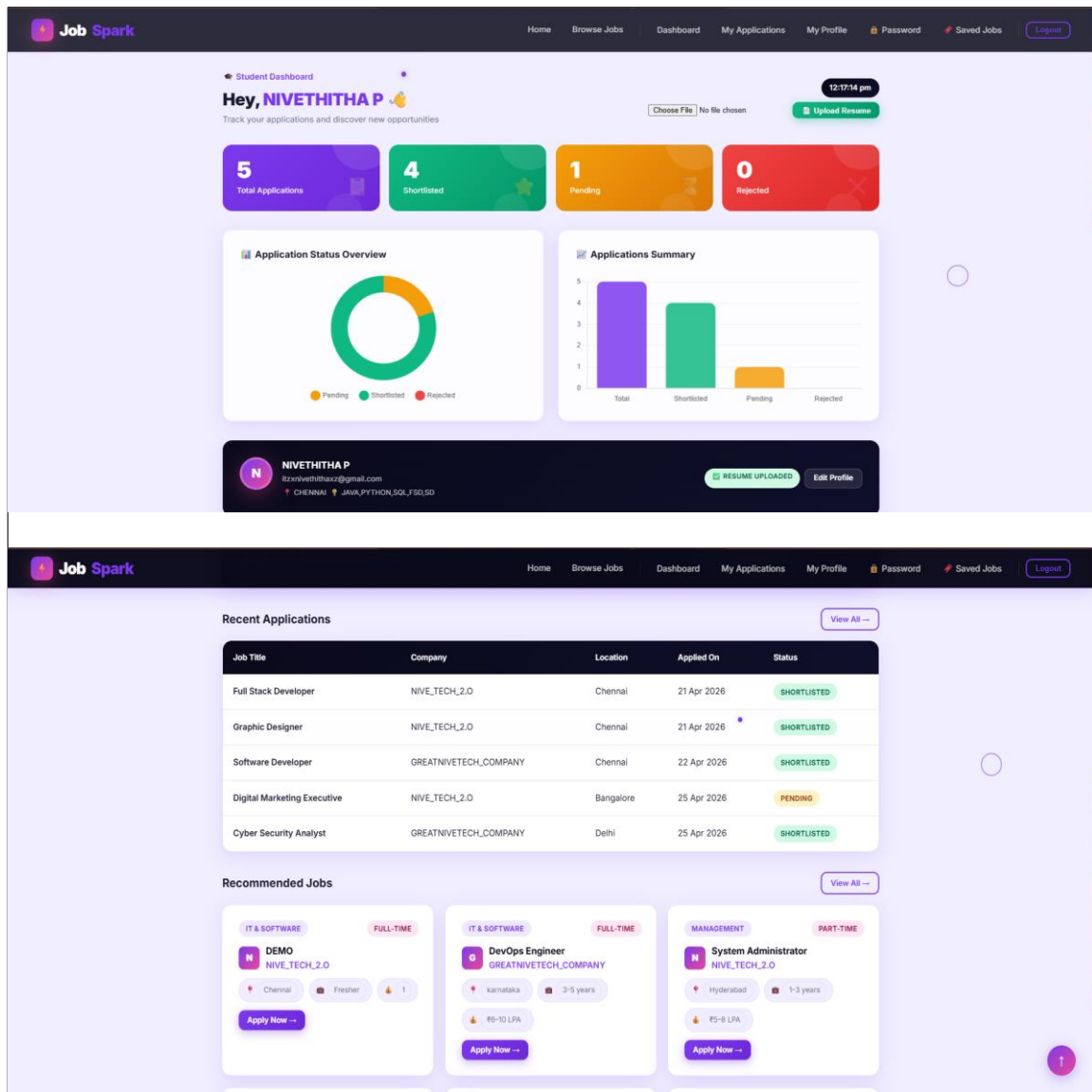


Figure 5.4: Student Dashboard with Application Statistics and Charts

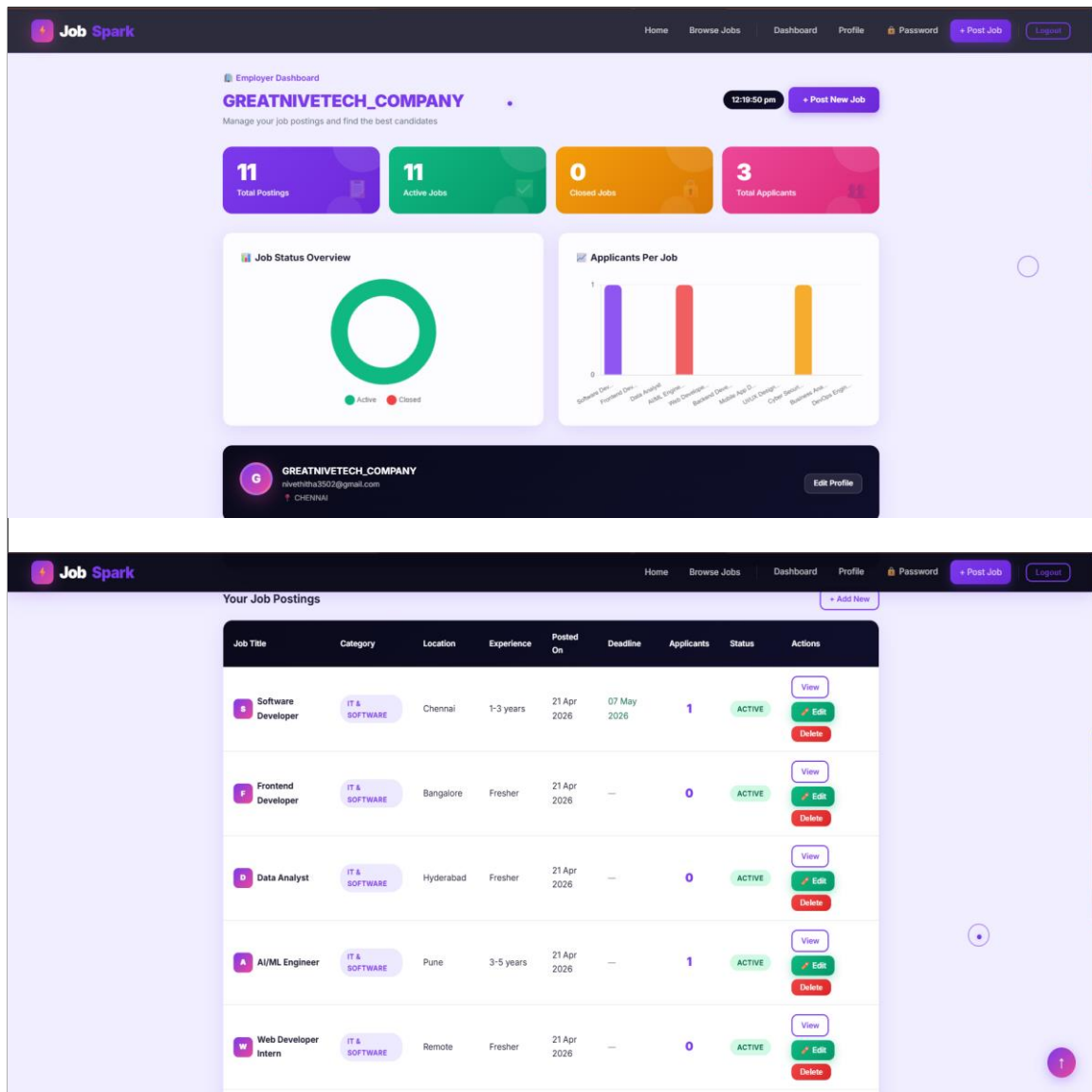


Figure 5.5: Employer Dashboard with Job Management Table and Analytics

Table 5.1: System Performance and Feature Analysis

Module	Feature	Observation / Result
Frontend	User Interface	Responsive glassmorphism design with smooth animations and particle background
Frontend	Form Validation	Prevents invalid inputs using @NotBlank and @Email annotations
Backend	Application Process	Successfully processes and stores all applications in MySQL
Backend	Email Notification	HTML email sent on Shortlist/Reject via Gmail SMTP asynchronously
Database	Data Storage	Efficient storage using four MySQL JPA tables with auto-created schema
System	Role-Based Access	Spring Security enforces URL-level role protection for all three roles
System	Performance	Fast response for all CRUD, search, and filter operations

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Job Portal Management System — JobSpark — was successfully designed and developed using HTML5, CSS3, JavaScript ES6, Thymeleaf, Spring Boot, Spring Security, Spring Data JPA, Hibernate, and MySQL. The system provides an efficient platform for students to discover and apply for jobs, and for employers to post listings and manage applicants, through a simple and interactive glassmorphism interface.

The application ensures proper role-based access control, BCrypt password encryption, real-time application status tracking, automated email notifications via Gmail SMTP, and Chart.js analytics dashboards for all user roles. The integration between the Thymeleaf frontend and the Spring Boot backend enables smooth communication and real-time data updates. Overall, the project achieves its objective of digitising the job portal process and reducing manual effort for both students and employers.

6.2 Future Enhancements

The current system can be further improved by adding advanced features and scalability options. Some possible enhancements include:

- Implementing REST API with JWT authentication for mobile application integration
- Integrating AI-based job recommendation algorithms using student skill profiles
- Adding cloud deployment on AWS or Railway for public internet accessibility
- Developing a dedicated Android and iOS mobile application consuming the REST API
- Migrating resume file storage to AWS S3 or Google Cloud Storage for scalability
- Implementing Spring Data Pageable for job listing pagination at scale
- Adding WebSocket-based real-time chat between employers and short-listed candidates

Chapter 7

SOURCE CODE

7.1 Source Code

```
1 package com.jobportal.model;
2 import jakarta.persistence.*;
3 import lombok.*;
4 import java.time.LocalDateTime;
5 import java.util.List;
6
7 @Entity
8 @Table(name = "users")
9 @Data @NoArgsConstructor
10 @AllArgsConstructor @Builder
11 public class User {
12     @Id
13     @GeneratedValue(strategy=GenerationType.IDENTITY)
14     private Long id;
15     private String fullName;
16     private String email;
17     private String password;
18     @Enumerated(EnumType.STRING)
19     private Role role;
20     private String phone;
21     private String location;
22     private String skills;
23     private String companyName;
24     private String resumePath;
25     @Column(updatable = false)
26     private LocalDateTime createdAt;
27     @PrePersist
28     protected void onCreate() {
29         createdAt = LocalDateTime.now();
30     }
```



```

31     @OneToMany(mappedBy = "employer",
32                cascade = CascadeType.ALL,
33                fetch = FetchType.LAZY)
34     private List<Job> postedJobs;
35     @OneToMany(mappedBy = "applicant",
36                cascade = CascadeType.ALL,
37                fetch = FetchType.LAZY)
38     private List<Application> applications;
39 }

```

Listing 7.1: User.java — JPA Entity Class

```

1 package com.jobportal.security;
2 import com.jobportal.service
3     .CustomUserDetailsService;
4 import lombok.RequiredArgsConstructor;
5 import org.springframework.context.annotation.*;
6 import org.springframework.security.config
7     .annotation.web.builders.HttpSecurity;
8 import org.springframework.security.config
9     .annotation.web.configuration
10    .EnableWebSecurity;
11 import org.springframework.security.crypto.bcrypt
12    .BCryptPasswordEncoder;
13 import org.springframework.security.crypto
14    .password.PasswordEncoder;
15 import org.springframework.security.web
16    .SecurityFilterChain;
17
18 @Configuration
19 @EnableWebSecurity
20 @RequiredArgsConstructor
21 public class SecurityConfig {
22     private final CustomUserDetailsService
23         userDetailsService;
24
25     @Bean
26     public PasswordEncoder passwordEncoder() {
27         return new BCryptPasswordEncoder();
28     }
29
30     @Bean
31     public SecurityFilterChain filterChain(
32         HttpSecurity http) throws Exception {
33         http
34             .csrf(csrf -> csrf.disable())
35             .authorizeHttpRequests(auth -> auth
36                 .requestMatchers("/", "/register",

```

```

37         "/login", "/css/**", "/js/**",
38         "/jobs/browse").permitAll()
39     .requestMatchers("/student/**")
40         .hasRole("STUDENT")
41     .requestMatchers("/employer/**")
42         .hasRole("EMPLOYER")
43     .requestMatchers("/admin/**")
44         .hasRole("ADMIN")
45     .anyRequest().authenticated()
46 )
47 .formLogin(form -> form
48     .loginPage("/login")
49     .successHandler(
50         customAuthSuccessHandler())
51     .failureUrl("/login?error=true")
52     .permitAll()
53 )
54 .logout(logout -> logout
55     .logoutUrl("/logout")
56     .logoutSuccessUrl("/login?logout=true")
57     .permitAll()
58 )
59 .userDetailsService(userDetailsService);
60 return http.build();
61 }
62
63 @Bean
64 public CustomAuthSuccessHandler
65     customAuthSuccessHandler() {
66     return new CustomAuthSuccessHandler();
67 }
68 }

```

Listing 7.2: SecurityConfig.java — Spring Security Configuration

```

1 package com.jobportal.service;
2 import com.jobportal.model.*;
3 import com.jobportal.repository
4     .ApplicationRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.stereotype.Service;
7 import java.util.*;
8
9 @Service
10 @RequiredArgsConstructor
11 public class ApplicationService {
12     private final ApplicationRepository
13         applicationRepository;

```

```

14 private final EmailService emailService;
15
16 public Application apply(Job job,
17     User applicant, String coverLetter) {
18     if (applicationRepository
19         .existsByJobAndApplicant(job, applicant))
20         throw new RuntimeException(
21             "Already applied for this job!");
22     Application app = Application.builder()
23         .job(job).applicant(applicant)
24         .coverLetter(coverLetter).build();
25     return applicationRepository.save(app);
26 }
27
28 public Application updateStatus(
29     Long applicationId,
30     ApplicationStatus status) {
31     Application app = applicationRepository
32         .findById(applicationId)
33         .orElseThrow(() ->
34             new RuntimeException(
35                 "Application not found"));
36     app.setStatus(status);
37     Application saved =
38         applicationRepository.save(app);
39     if (status == ApplicationStatus.SHORTLISTED
40         || status ==
41         ApplicationStatus.REJECTED) {
42         emailService.sendApplicationStatusEmail(
43             app.getApplicant().getEmail(),
44             app.getApplicant().getFullName(),
45             app.getJob().getTitle(),
46             app.getJob().getEmployer()
47                 .getCompanyName(),
48             status.name());
49     }
50     return saved;
51 }
52
53 public List<Application> getAllApplications() {
54     return applicationRepository.findAll();
55 }
56 }

```

Listing 7.3: ApplicationService.java — Email Notification on Status Update

```

1 package com.jobportal.service;
2 import com.jobportal.model.*;

```

```

3 import com.jobportal.repository
4     .SavedJobRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.stereotype.Service;
7 import org.springframework.transaction
8     .annotation.Transactional;
9 import java.util.List;
10
11 @Service
12 @RequiredArgsConstructor
13 public class SavedJobService {
14     private final SavedJobRepository
15         savedJobRepository;
16
17     @Transactional
18     public boolean toggleSaveJob(
19         User student, Job job) {
20         if (savedJobRepository
21             .existsByStudentAndJob(student, job)) {
22             savedJobRepository
23                 .deleteByStudentAndJob(student, job);
24             return false;
25         } else {
26             SavedJob savedJob = SavedJob.builder()
27                 .student(student).job(job).build();
28             savedJobRepository.save(savedJob);
29             return true;
30         }
31     }
32
33     public List<SavedJob> getSavedJobs(
34         User student) {
35         return savedJobRepository
36             .findByStudent(student);
37     }
38
39     public boolean isJobSaved(
40         User student, Job job) {
41         return savedJobRepository
42             .existsByStudentAndJob(student, job);
43     }
44 }

```

Listing 7.4: SavedJobService.java — Job Bookmark Toggle Feature

Chapter 8

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

8.1 Mapping of the Project to Complex Engineering Problem (CEP)

The JobSpark Job Portal Management System addresses several attributes of Complex Engineering Problems as defined by engineering education standards. The following table maps each attribute to specific characteristics and knowledge areas demonstrated by this project.

Table 8.1: Mapping of Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKS	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge in Spring Boot, Spring Security, JPA/Hibernate, MySQL, and Gmail SMTP for building a secure multi-role web system.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances security and usability — BCrypt hashing must not slow UX, and role-based restrictions must not hinder legitimate access.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Involves MVC architecture design, JPA entity relationship modelling, and async email service integration with real-time application tracking.
WP4	Familiarity	Partially new problems	WK4, WK8	Glassmorphism UI with Chart.js analytics and async email notifications introduces challenges beyond traditional web systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom JPQL search queries, custom Spring Security handlers, and a custom HTML email template builder.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Handles requirements of students (job seekers), employers (recruiters), and administrators ensuring secure and role-appropriate interactions.
WP7	Interdependencies	System-level integration	WK3, WK5, WK6	Integrates Thymeleaf views, Spring MVC controllers, service logic, JPA repositories, MySQL, and Gmail SMTP into one cohesive system.

8.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 8.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	Spring Boot, Spring Security, JPA, Hibernate, Thymeleaf, MySQL, Chart.js, Gmail SMTP, JavaScript ES6, CSS3 Glassmorphism.
EA2	Level of Interactions	Complex module interactions	Student application module, employer job management, admin oversight, email service, and Spring Security all interact simultaneously.
EA3	Innovation	Modern technologies	Chart.js dashboards, canvas particle animations, 3D card tilt effects, glassmorphism UI, confetti animations, and async email dispatch demonstrate innovation.
EA4	Societal Impact	Impact on users and society	Improves employment accessibility for students, reduces manual recruitment overhead for employers, and promotes digital transformation of hiring.
EA5	Familiarity	Beyond standard practices	Combines role-based security, async email processing, real-time animations, job expiry, cover letter modals, and bookmarks beyond typical CRUD apps.

8.3 SDG Mapping

Table 8.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Provides a platform for students and fresh graduates to discover jobs, supporting career readiness through accessible digital tools at no cost.
SDG 8	Decent Work and Economic Growth	Directly connects job seekers with employers, reducing unemployment gaps and enabling efficient talent discovery which contributes to economic growth.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in recruitment using Spring Boot, Chart.js analytics, and automated email — demonstrating modern software innovation.
SDG 10	Reduced Inequalities	Provides a completely free platform with no premium paywalls, ensuring equal access to job opportunities for students from all economic backgrounds.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure data management through BCrypt hashing, CSRF protection, and role-based access control protecting all stakeholders.

References

- [1] VMware Inc. (2024). Spring Boot Documentation. *Spring Boot Official Reference*, 1(1), pp. 1–20.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [2] VMware Inc. (2024). Spring Security Reference Documentation. *Spring Security Official Docs*, 2(1), pp. 1–18.
<https://docs.spring.io/spring-security/reference/index.html>
- [3] VMware Inc. (2024). Spring Data JPA Documentation. *Spring Data Official Reference*, 3(2), pp. 1–15.
<https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- [4] The Thymeleaf Team (2024). Thymeleaf Documentation. *Thymeleaf Official Docs*, 4(1), pp. 1–12.
<https://www.thymeleaf.org/documentation.html>
- [5] Oracle Corporation (2024). MySQL 8.0 Reference Manual. *MySQL Official Documentation*, 5(3), pp. 1–20.
<https://dev.mysql.com/doc/refman/8.0/en/>
- [6] Chart.js Contributors (2024). Chart.js Documentation. *Chart.js Official Docs*, 6(1), pp. 1–10.
<https://www.chartjs.org/docs/latest/>
- [7] The Lombok Authors (2024). Project Lombok Documentation. *Lombok Official Docs*, 7(1), pp. 1–8.

<https://projectlombok.org/features/>

- [8] The Apache Software Foundation (2024). Apache Maven Documentation. *Maven Official Guides*, 8(2), pp. 1–10.

<https://maven.apache.org/guides/>

- [9] Info Edge India Ltd. (2024). Naukri.com — India’s No.1 Job Portal. *Naukri Technical Report*, 9(1), pp. 1–5.

<https://www.naukri.com>

- [10] LinkedIn Corporation (2024). LinkedIn Jobs Platform. *LinkedIn Engineering Journal*, 10(2), pp. 1–8.

<https://www.linkedin.com/jobs/>

- [11] Indeed Inc. (2024). Indeed Job Search Platform. *Indeed Technical Documentation*, 11(1), pp. 1–6.

<https://www.indeed.com>

- [12] Internshala (2024). Internshala — Internships and Jobs for Students. *Internshala Platform Guide*, 12(1), pp. 1–5.

<https://www.internshala.com>